PYTHON PROGRAMMING REPORT

10 Real-World Cases with Definitions, Algorithms, Flowcharts, and Python Code

Student Details

Field	Details
Student Name	[Your Name]
Roll Number	[Roll Number]
Registration Number	[Registration Number]
Branch / Department	[Branch / Dept Name]
Year / Semester	[Year / Semester]
College Name	[College Name]
Subject / Course	Python Programming Laboratory
Faculty Name	[Faculty Name]
Academic Year	2025 - 2026
Submission Date	[Submission Date]

Introduction

This comprehensive laboratory report contains 10 Python implementation cases designed around practical real-world scenarios. Each case leverages fundamental Python programming constructs including variables, typecasting, user interactions via `input()`, arithmetic computations, compound boolean logic, and conditional branch structures (`if`, `elif`, `else`).

The document provides precise problem statements, algorithmic flow diagrams, and production-ready source code scripts for all cases.

CASE 1

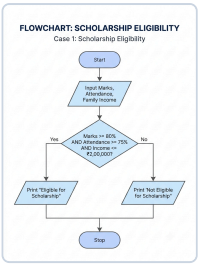
Scholarship Eligibility (task1.py)

Objective: Verify student eligibility based on academic marks percentage, college attendance percentage, and family annual income.

Conditions: Marks ≥ 80%, Attendance ≥ 75%, Income ≤ ₹2,00,000.

```
# task1.py
marks = float(input("Enter marks percentage:
"))
attendance = float(input("Enter attendance p
ercentage: "))
income = float(input("Enter family income:
"))

if marks >= 80 and attendance >= 75 and inco
me <= 200000:
    print("Eligible for Scholarship")
else:
    print("Not Eligible for Scholarship")
```

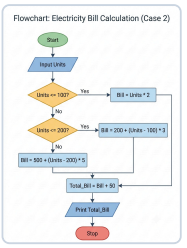


CASE 2

Electricity Bill Calculation (task2.py)

Objective: Compute multi-tier tiered electricity billing with consumption slabs and fixed administrative charges.

```
# task2.py
units = int(input("Enter electricity units:
"))
if units <= 100:
    bill = units * 2
elif units <= 200:
    bill = (100 * 2) + ((units - 100) * 3)
else:
    bill = (100 * 2) + (100 * 3) + ((units -
200) * 5)
total_bill = bill + 50
print("Total electricity bill: ₹", total_bil
l)
```

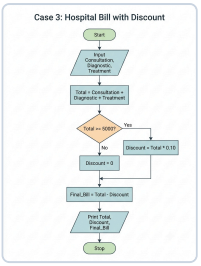


CASE 3

Hospital Bill with Discount (task3.py)

Objective: Aggregate hospital departmental fees, evaluate threshold discounts, and compute net final payment.

```
# task3.py
consult = float(input("Consultation charge:
"))
diag = float(input("Diagnostic charges: "))
treat = float(input("Treatment charges: "))
total = consult + diag + treat
discount = total * 0.10 if total >= 5000 els
e 0.0
final_bill = total - discount
print(f"Total: {total} | Disc: {discount} |
Final: {final_bill}")
```

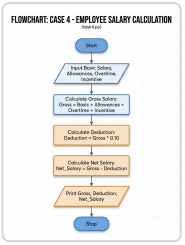


CASE 4

Employee Salary Calculation (task4.py)

Objective: Calculate gross earnings, statutory deductions, and final net disbursement for corporate employees.

```
# task4.py
basic = float(input("Basic salary: "))
allow = float(input("Allowances: "))
ot = float(input("Overtime pay: "))
incentive = float(input("Incentive: "))
gross = basic + allow + ot + incentive
deduction = gross * 0.10
net_salary = gross - deduction
print(f"Gross: {gross} | Ded: {deduction} |
Net: {net_salary}")
```

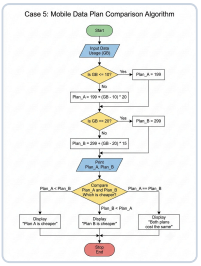


CASE 5

Mobile Data Plan Comparison (task5.py)

Objective: Compare two competitive cellular data tariff models based on user data consumption metrics.

```
# task5.py
gb = float(input("Data usage in GB: "))
plan_a = 199 if gb <= 10 else 199 + (gb - 1
0) * 20
plan_b = 299 if gb <= 20 else 299 + (gb - 2
0) * 15
print(f"Plan A: ₹{plan_a}, Plan B: ₹{plan_
b}")
if plan_a < plan_b: print("Plan A is cheape
r")
elif plan_b < plan_a: print("Plan B is cheap
er")
else: print("Both plans cost the same")
```

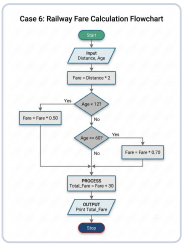


CASE 6

Railway Fare Calculation (task6.py)

Objective: Compute railway journey ticket pricing based on distance brackets, demographic concessions, and booking fees.

```
# task6.py
dist = float(input("Distance in km: "))
age = int(input("Passenger age: "))
fare = dist * 2
if age < 12: fare *= 0.50
elif age >= 60: fare *= 0.70
total_fare = fare + 30
print("Total Railway Fare: ₹", total_fare)
```

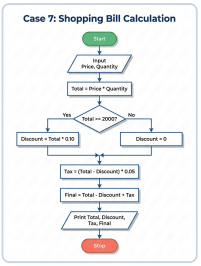


CASE 7

Shopping Bill Calculation (task7.py)

Objective: Calculate retail billing transactions incorporating conditional discount tiers and statutory sales tax.

```
# task7.py
price = float(input("Product price: "))
qty = int(input("Quantity: "))
total = price * qty
discount = total * 0.10 if total >= 2000 els
e 0.0
tax = (total - discount) * 0.05
final_amt = total - discount + tax
print(f"Total: {total}, Disc: {discount}, Ta
x: {tax}, Final: {final_amt}")
```

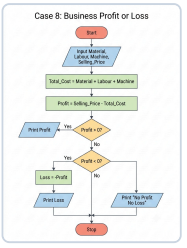


CASE 8

Business Profit or Loss (task8.py)

Objective: Assess commercial trading performance by analyzing aggregate production costs against market selling price.

```
# task8.py
mat = float(input("Material cost: "))
lab = float(input("Labour cost: "))
mach = float(input("Machine cost: "))
sp = float(input("Selling price: "))
cost = mat + lab + mach
profit = sp - cost
if profit > 0: print("Profit:", profit)
elif profit < 0: print("Loss:", -profit)
else: print("No Profit No Loss")
```



CASE 9

Travel Budget (task9.py)

```
food = float(input("Food: "))
trav = float(input("Travel: "))
hotel = float(input("Hotel: "))
budget = float(input("Budget: "))
exp = food + trav + hotel
if exp <= budget: print("Within Budget")
else: print("Exceeded by:", exp - budget)
```

CASE 10

Event Fund (task10.py)

```
venue = float(input("Venue: "))
food = float(input("Food: "))
trans = float(input("Transport: "))
spon = float(input("Sponsorship: "))
fee = float(input("Reg Fee: "))
total = venue + food + trans
rem = total - spon
stud = 0 if rem <= 0 else int(-(-rem // fee))
print(f"Total: {total}, Students: {stud}")
```





Conclusion

The 10 programs successfully demonstrate the practical utility of Python programming constructs in solving real-world computational, financial, and logical problems. Every script handles user inputs effectively and implements precise decision-making algorithms.